32 字串匹配

Part VII Selected Topics

本章定位

字串匹配把已知相等的前後綴轉成可重用狀態；本章從逐字比較與模數碰撞，推導有限自動機與KMP的失敗鏈、正確性及線性總成本。

前置知識

模式前綴、雜湊與有限自動機。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 定義 valid shifts 與 prefix 問題。
- 比較 naive、Rabin-Karp、automaton 與 KMP。
- 證明 KMP 前綴函數與線性總時間。
- 32.1 The naive string-matching algorithm
- 32.2 The Rabin-Karp algorithm
- 32.3 String matching with finite automata
- 32.4 The Knuth-Morris-Pratt algorithm

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

32.1 The naive string-matching algorithm

文字（text）T 的長度為 n，模式（pattern）P 的長度為 m；字元來自有限 alphabet Σ，相等比較採 O(1)。本章字元位置一律從1開始，shift s 從0開始：合法位移滿足 $0 \le s \le n-m$ 且對每個 $j=1,\dots,m$ 都有 $P[j]=T[s+j]$。輸出所有位移，允許重疊。

P_k 表示 $P[1..k]$，$P_0=\varepsilon$ 為空字串。Prefix 是前綴，suffix 是後綴；proper prefix/suffix 不得等於整個字串。Border 指同時為前綴與後綴的字串。

若 $m=0$，規格定為輸出$0,\dots,n$這 $n+1$ 個邊界；若 $m>n$，輸出空集合。後續主演算法先處理這兩種情形，才假設$1 \le m \le n$，避免讀取不存在的 $P[1]$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐字比較：不漏位移也不接受錯字

```
for s=0..n-m:
    j=1
    while j<=m and P[j]==T[s+j]: j++
    if j==m+1: output s
```

內圈每次檢查前，P[1..j−1]=T[s+1..s+j−1]。相等才延伸不變量；不等就知道此位移不合法；j=m+1 表示全部相等。外圈恰枚舉所有合法範圍，因此聲稱的每個匹配正確，真正匹配也不會漏掉。

共有 n−m+1 個窗口，每個至多 m 次比較，最壞 Θ((n−m+1)m)。取 T=a^n、P=a^m，每個窗口都用滿 m 次，證明此界可達；即使沒有碰撞，也可能有很多真正匹配。額外工作空間 O(1)，不含輸出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一文字的所有窗口與比較次數

T=ababaca、P=aba。遇到第一個不相等就停止：

s	窗口	依序比較	次數	結果
0	aba	a=a，b=b，a=a	3	輸出0
1	bab	a≠b	1	拒絕
2	aba	a=a，b=b，a=a	3	輸出2
3	bac	a≠b	1	拒絕
4	aca	a=a，b≠c	2	拒絕

共10次比較。s=0與s=2共用文字位置3，仍是兩個合法匹配。Naive 重新檢查同一字元對齊；KMP 將利用已知相等的區段排除不可能的位移。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

32.2 The Rabin-Karp algorithm

將字元單射編為0,…,d−1，以 radix d 表示長 m 字串。其整數值 H(X)=Σ_(j=1)^m X[j]d^(m−j)，但 m 位數的大整數運算不能當 O(1)。因此保留 h_q(X)=H(X) mod q，q≥2，並在 hash 相等時逐字驗證。

令 p=h_q(P)、t_s=h_q(T[s+1..s+m])、h=d^(m−1) mod q。由移除最高位、左移一位、加入新低位，得到

$$t_{s+1} = \big(d(t_s - T[s+1]h) + T[s+m+1]\big) \bmod q.$$

此恆等式對任意 q≥2 成立，不必是質數；選質數有利於某些隨機化分析，卻不自動保證固定字串的碰撞率為1/q。實作須將負餘數正規化為0,…,q−1，並讓乘法中間值不溢位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hash 只排除，字元驗證才接受

```
h=1; p=0; t=0
for j=1..m-1: h=(d*h) mod q
for j=1..m:
    p=(d*p+P[j]) mod q
    t=(d*t+T[j]) mod q
for s=0..n-m:
    if p==t and P[1..m]==T[s+1..s+m]: output s
    if s<n-m:
        t=(d*(t-T[s+1]*h)+T[s+m+1]) mod q
```

Horner 法與 rolling 恆等式歸納保證 t 始終是當前窗口 hash。真正相等的字串必有相同 hash，所以不會漏報；hash 不同可拒絕，hash 相同再比較原字串，所以不會誤報。碰撞影響時間，不影響這個驗證版本的正確性。

若省略原字串驗證，則變成可能誤報的演算法，不能沿用「總是正確」的結論。

一次真正匹配、一次碰撞、再一次匹配

取 T=122312、P=12、d=10、q=11。h=10，p=12 mod11=1：

s	窗口	t_s	計算下一個 hash	字元驗證
0	12	1	(10(1−1・10)+2) mod11=0	12=12，輸出0
1	22	0	(10(0−2・10)+3) mod11=1	hash不同，略過
2	23	1	(10(1−2・10)+1) mod11=9	12≠23，拒絕碰撞
3	31	9	(10(9−3・10)+2) mod11=1	hash不同，略過
4	12	1	最後窗口，不再讀新字元	12=12，輸出4

Spurious hit（假命中）是 hash 相等而字串不同；此例23−12=11，正好被模數消掉。即使 q 是質數，碰撞仍可發生。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

一次真正匹配、一次碰撞、再一次匹配 (續)

窗口 12 → 22：d=10，q=11，h=10

舊 hash
$t_0=1$

→

移除最高位 1
$1-1\cdot10=-9$

→

乘10、加入2
$-9\cdot10+2=-88$

$-88 \bmod 11=0$

窗口 12：hash=1=p
逐字驗證 12=12
接受 shift 0 與 4

窗口 23：hash=1=p
逐字驗證 12≠23
拒絕 shift 2：$23-12=11$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成本取決於真匹配與假命中

採 word-RAM：字元、q、所有模乘中間值能在 O(1) word 操作完成，或提供 O(1) 模乘。若直接算 d(t−out・h)，也必保證其中間值可容納；可先把差正規化後再乘 d，使中間值少於 dq。任意精度版本須另乘位元運算成本。

預處理 Θ(m)，rolling 掃描 Θ(n−m+1)。設真匹配數 z、假命中數 f，每次驗證至多 m 字元，總時間 `O(n+m+m(z+f))`。

最壞 z 或 f 可達 Θ(n−m+1)，所以最壞仍 Θ((n−m+1)m)。例如全 a 文字與全 a 模式只有真命中，照樣反覆驗證；不能把退化全歸因於選錯 hash。輸出匹配本身要 Ω(z)，本版本另外花 m 次字元驗證每個真命中。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

有證明的隨機 Hash：明寫亂數與條件

另一種可分析的版本：固定質數 q>d−1，字元編碼不同即在域 F_q 不同；在輸入固定後，獨立均勻選 base b∈F_q，以 ΣX[j]b^(m−j) mod q 作 hash。Rolling 公式以 b 取代 d，b=0也依恆等式處理，m=1時 h=b^0=1。

對固定相異字串 X,Y，差多項式 D(b) 非零、次數至多 m−1。非零域上多項式至多有其次數個根：若 r 是根，因式定理提出(b−r)，再對次數歸納。因此

$\Pr[h_b(X)=h_b(Y)] \le \min(1,(m-1)/q)$。

對每個非匹配窗口的碰撞指示變數取期望再相加，不需要窗口事件獨立，得到 E[f]≤(n−m+1−z)(m−1)/q。故期望時間為 O(n+m+mz+mnm/q) 的寬鬆界；選 q 足夠大才有低假命中成本，真匹配的 mz 仍在。q 的位元長度與模乘成本也必計入，不能任意增大後仍無條件稱 O(1)。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

32.3 String matching with finite automata

有限自動機由(Q,q_0,A,Σ,δ)組成：有限狀態集合、初態、接受狀態集、字母表及每個狀態／字元對的轉移函數。匹配器令 $Q=\{0,\cdots,m\}$、$q_0=0$、$A=\{m\}$。

定義 $\sigma(x)=\max\{k\le m：P_k$ 是 x 的後綴$\}$。空字串總是後綴，所以最大值存在。狀態 $q=\sigma$(目前已讀文字)，意義是「現在可延伸的最長匹配前綴」，不是已經找到幾次完整匹配。

令 $\delta(q,a)=\sigma(P_q\ a)$。讀入一個字元後做一次轉移；當狀態等於 m 就輸出目前結尾所決定的位移。接受狀態並非停止狀態，仍有對每個 a 的轉移，才能找到重疊匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何只記最長前綴長度就夠

要證明 δ(σ(x),a)=σ(xa)。令 q=σ(x)、r=σ(xa)。若 r>0，把匹配的最後一字移去，P_(r−1) 是 x 的後綴，因此 r−1≤q。P_q 也是 x 的後綴；兩個後綴按長度包含，故 P_(r−1) 也是 P_q 的後綴，再接 a，P_r 就是 P_q a 的後綴。因此 r≤σ(P_q a)。r=0時此不等式也成立。

反過來，P_q a 本身是 xa 的後綴，任何同時為 P 前綴的 P_q a 後綴也必為 xa 後綴，故 σ(P_q a)≤σ(xa)=r。兩向合併得等式。

由初態 σ(ε)=0歸納，每次轉移後 q=σ(T[1..i])。q=m 當且僅當 P 是 T[1..i] 的後綴，恰好等價於 shift i−m 匹配；這證明接受判斷的必要性與充分性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整轉移表與重疊匹配

P=abab，Σ={a,b,c}，每格是 δ(q,字元)：

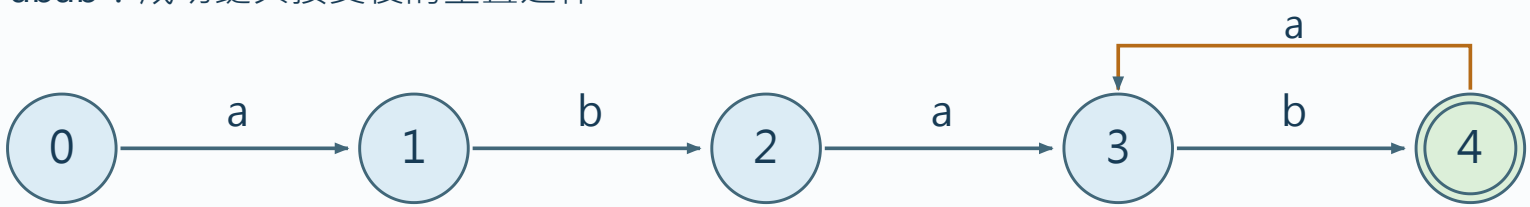
q	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	3	0	0

T=ababab 的狀態依序1,2,3,4,3,4，在 i=4與6輸出 shift0與2。δ(4,a)=3，因 ababa 的最長可用後綴是 aba。若匹配後無條件清零，就會錯過第二次匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整轉移表與重疊匹配 （續）

P=abab：成功鏈與接受後的重疊延伸



完整轉移表見正文；此圖只畫成功鏈與 δ(4,a)=3。

T[i]	a	b	a	b	a	b
狀態	1	2	3	4	3	4

讀到第4與第6字接受：shift 0、2；完整匹配之後不清零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

建表成本不能混入掃描成本

直接按 δ 的定義，對每個 q,a 從 min(m,q+1) 往下試 k，逐字驗證 P_k 是否為 P_q a 的後綴，可花 $O(m^2)$ 每格，總 $O(m^3|\Sigma|)$。這是正確但慢的建法；掃描仍是 Θ(n)。

有了下一節的 prefix function π，可按 q 遞增建表：

```
for a in Σ: δ(0,a) = (1 if a==P[1] else 0)
for q=1..m:
    for a in Σ:
        if q<m and a==P[q+1]: δ(q,a)=q+1
        else: δ(q,a)=δ(π[q],a)
```

Mismatch 時，下一個候選只能是 P_q 的 proper border；最長的長度為 π[q]。所有較短候選都在它的後綴內，所以同一字元轉移可重用其表格。π[q]<q使依賴格已算完。建 π 要 Θ(m)，建表 $\Theta(m|\Sigma|)$ 時間與空間；只有在可 O(1) 索引 alphabet 的模型下，每字元查表才是 O(1)。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

32.4 The Knuth-Morris-Pratt algorithm

Prefix function（前綴函數）定為 π[q]=max{k<q：P_k 是 P_q 的後綴}，q=1,⋯,m；因此 π[1]=0且π[q]<q。它只依模式，與文字無關。

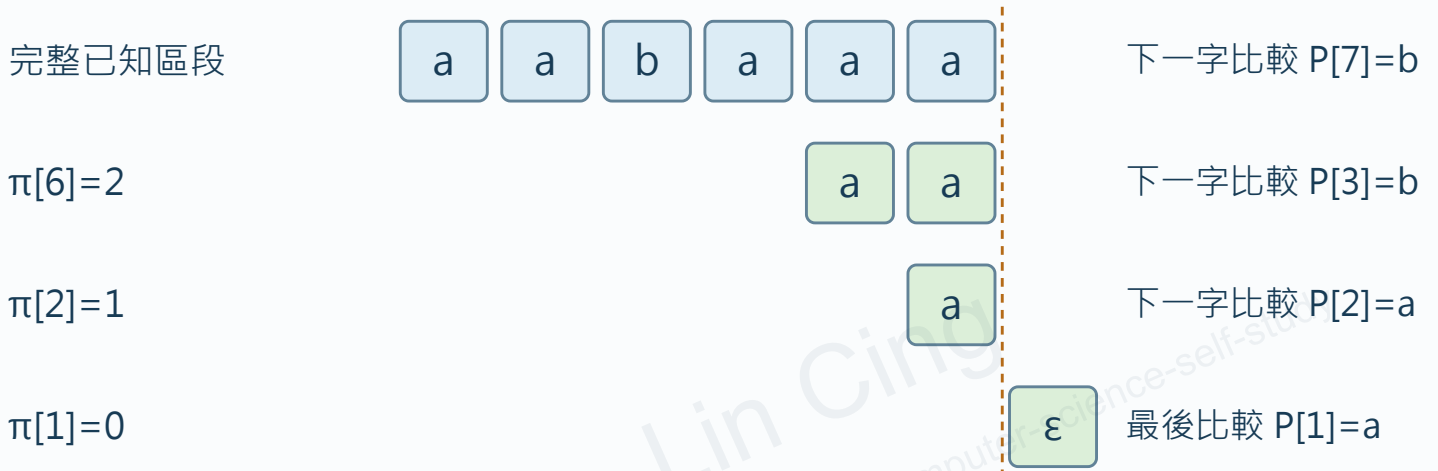
對 P=aabaaab，π=[0,1,0,1,2,2,3]。P_6=aabaaa 的 proper borders 是 aa、a、ε，長度鏈為 6→2→1→0。6本身只是搜尋的起點，不算 proper border。

KMP不儲存整張 δ 表。下一字元無法延伸時，就沿 π 鏈找下一個可能延伸的前綴。每次減少候選長度，同一個新文字字元保持不動，已讀過的文字不重讀。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

32.4 The Knuth-Morris-Pratt algorithm （續）

P=aabaaab，已知文字後綴 aabaaa（q=6）



新字元是 c：6→2→1→0，所有比較皆失敗；同一個 c 尚未向前移動。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

失敗鏈列出了全部候選，不會跳過可行長度

令 B(q) 為 P_q 的所有 proper border 長度。若 b=π[q]>0，任何 k<b 都有

$k \in B(q) \Leftrightarrow k \in B(b)$。

證明：P_b 是 P_q 的後綴，因此它的長 k 後綴就是 P_q 的長 k 後綴；而 P_k 是兩者共用的前綴。任一方向的前後綴相等條件因此相同。b 是最長 proper border，不可能有 b<k<q 的候選。若 b=0，沒有正長度的 proper border。

所以 B(q)={b}∪B(b)，依嚴格遞減長度歸納，反覆 π[q]、π[π[q]]、…、0恰好列出所有 proper border。這是 fallback 安全的原因；不是因「短一點大概會匹配」。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Prefix Function 的逐項建構

```
π[1]=0; k=0
for i=2..m:
    while k>0 and P[k+1]!=P[i]: k=π[k]
    if P[k+1]==P[i]: k++
    π[i]=k
```

在處理 i 前，k=π[i−1]。P_i 若有長 r>0 的 proper border，去掉最後字元後，長 r−1 必是 P_(i−1) 的 proper border；反之，只要該 border 之後的 P[r] 等於 P[i]，就能延長成長 r 的 border。

故由最大候選 π[i−1] 開始，沿完整失敗鏈遞減，第一個可延伸者產生最大合法 r；到0仍不相等則答案0。先前已算正確的 π 值只在較小索引被使用，歸納成立。因 π[i−1]≤i−2，這一步不會把整個 P_i 自身當 proper border。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一模式的建表全軌跡

P=aabaaab：

i	P[i]	初始k	比較與fallback	π[i]
1	a	—	單字元無非空proper border	0
2	a	0	P[1]=a，延長	1
3	b	1	P[2]=a≠b；1→0；P[1]仍不符	0
4	a	0	P[1]=a	1
5	a	1	P[2]=a	2
6	a	2	P[3]=b≠a；2→1；P[2]=a	2
7	b	2	P[3]=b	3

π[6]=2不是3：aabaaa的末三字aaa不等於前三字aab。建表不是在不相等時一律 k−−，而是只保留已知可行的 border。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP 掃描與匹配後的狀態

```
π=prefix_function(P); q=0
for i=1..n:
    while q>0 and P[q+1]!=T[i]: q=π[q]
    if P[q+1]==T[i]: q++
    if q==m:
        output i-m
        q=π[m]
```

在接受檢查時，q=σ(T[1..i])，與自動機相同。若未達 m，便保存此 q 供下輪使用；若達 m，先輸出，再保存 π[m]，即已讀文字中最長的 proper 模式前綴後綴。下一輪延伸新匹配只需要 proper border，因目前完整長 m 前綴不可能再增至 m+1。

Mismatch 的所有可行較短候選恰為失敗鏈，第一個能與 T[i] 延伸者就是 δ 的結果；若無候選則為0。這同時證明不漏任何匹配，以及讀取 P[q+1] 前永遠 q<m，不會越界。

完整匹配之後仍能接重疊匹配

P=aabaaab、π[7]=3，取 T=aabaaabaaab：

i	T[i]	比較前q	fallback後／延伸後q	輸出後保存q
1	a	0	1	1
2	a	1	2	2
3	b	2	3	3
4	a	3	4	4
5	a	4	5	5
6	a	5	6	6
7	b	6	7，輸出0	3
8	a	3	4	4
9	a	4	5	5

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整匹配之後仍能接重疊匹配 （續）

i	T[i]	比較前q	fallback後／延伸後q	輸出後保存q
10	a	5	6	6
11	b	6	7，輸出4	3

兩次匹配共用位置5–7的 aab。清零會丟掉這段已知相等的證據。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個字元可多次 fallback，總成本仍線性

狀態 q=6、下一字元 c 時，依序比較 P[7]=b、P[3]=b、P[2]=a、P[1]=a，都不等 c；鏈6→2→1→0，最後保存0。同一個 c 被比較多次，但文字索引 i 沒有倒退。

掃描中 q 初始0，每處理一字最多增加1，總增加量≤n。每次 while body 至少減1，輸出後的 q←π[m] 也只減不增；q 從不負。因此 while body 的總次數≤n，另加每字常數次外圈檢查及輸出，匹配 Θ(n)。相同記帳適用建 π：總增加≤m−1，while body總次數≤m−1，建表 Θ(m)。

總時間 Θ(n+m)，額外工作空間 Θ(m)，不含輸出。這是整段序列的界，不是說每個字元最壞 O(1)；若需每字固定最壞轉移時間，可改付完整自動機表的空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四種方法使用了哪些假設

方法	前處理	掃描最壞時間	模式額外空間
Naive	無	Θ((n−m+1)m)	O(1)
RK，hash相等後驗證	Θ(m)	Θ((n−m+1)m)	O(1)
完整自動機，π建表	Θ(m	Σ	)
KMP	Θ(m)	Θ(n)	Θ(m)

本表採1≤m≤n、固定成本字元及word模算術，預處理與掃描分開列。RK期望快需要額外亂數／碰撞條件；KMP線性不需要隨機輸入或隨機模式。Unicode實作要先選定匹配單位是byte、code point或grapheme cluster；本章不把不同編碼的字元序列自動視為相同字串。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 32。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study